Computer Architecture Lab 5

Jonas Van Nieuwenhuyze, Gil Nimmegeers

1 Introduction

When data needs to be transferred to or from the GPU, the CPU is idle by default, thus the memory copying is synchronous. A GPU kernel does not block the CPU, so it is asynchronous by default. A full stack of copying memory and calling the kernel can be made asynchronous so that the CPU and GPU can continue working concurrently.

2 Asynchronous Operations

Asynchronous operations allow the CPU and GPU to work concurrently, improving overall performance by overlapping computation and data transfers. With synchronous operations, the CPU initiates a task on the GPU and waits until the GPU completes it before proceeding. This blocking behavior can lead to underutilization of resources. Asynchronous operations, on the other hand, enable the CPU to continue its tasks while the GPU performs its computations. This concurrency maximizes resource utilization and can lead to significant performance gains.

2.1 cudaMemcpyAsync

One common asynchronous operation is cudaMemcpyAsync, which is similar to the synchronous cudaMemcpy but does not block the CPU. Instead of waiting for the data transfer to complete, cudaMemcpyAsync returns immediately, allowing the CPU to continue executing other instructions.

2.2 Streams

To manage and coordinate asynchronous operations, CUDA provides the concept of streams. Streams are sequences of operations that execute asynchronously with respect to one another. By assigning operations to different streams, you can overlap their execution, leading to better resource utilization and performance.

3 Results

For small array sizes, asynchronous operations show marginal speed improvements over synchronous ones. This is primarily because the array preparation phase is quick, minimizing the CPU's benefit from starting early. However,

as array sizes increase (as depicted in Figure 1), the time gap between asynchronous and synchronous operations widens significantly. This disparity arises due to the increased time required for array preparation. Figure 2 illustrates this growing gap as array size increases.

Figure 1: CPU time compared to large array size

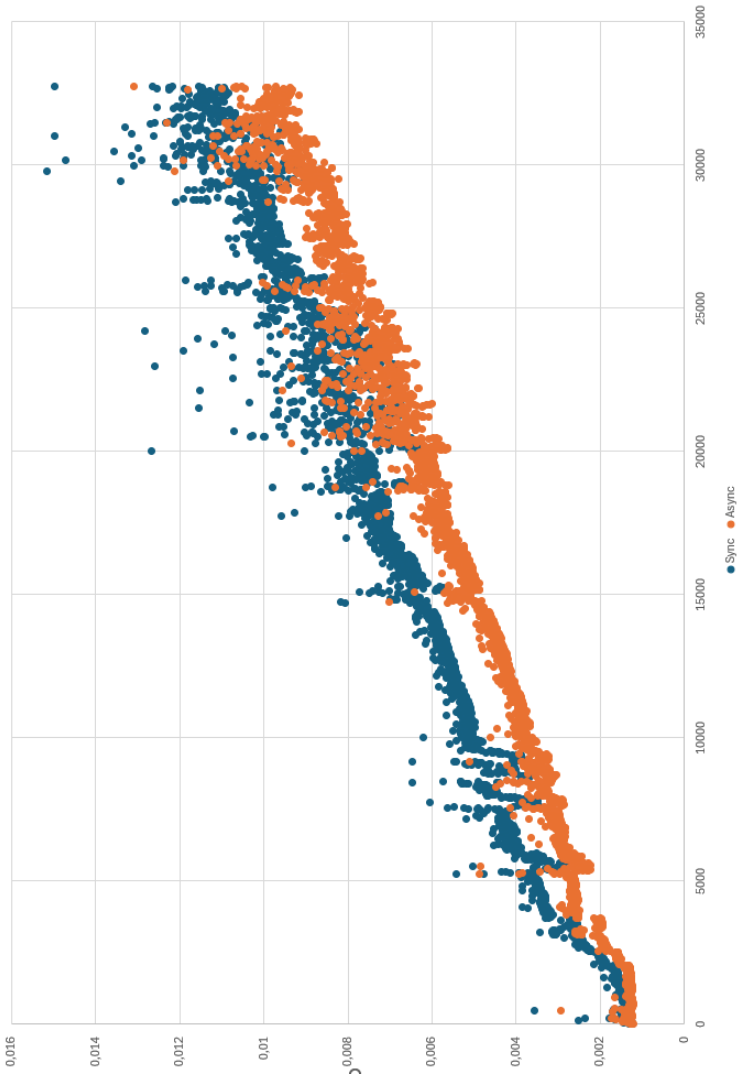


Figure 2: CPU time compared to low array sizes

